



Trabalho de Projeto

1ª Fase

1 Descrição Geral

A componente teórico-prática da disciplina de sistemas distribuídos consiste no desenvolvimento de quatro projetos, utilizando a linguagem de programação C [1], sendo que a realização de cada um deles é necessária para a realização do projeto seguinte. Por essa razão, **é muito importante que consigam cumprir os objetivos de cada projeto, de forma a não hipotecar os projetos seguintes.**

O projeto consiste em implementar um sistema de gestão de inventário para um conjunto de um stand automóvel, utilizando uma lista encadeada para armazenar a informação dos carros (ano, preço, marca, modelo, combustível), onde o ano é um número inteiro, o preço é um float, a marca e o combustível são definidos por enums (enumeração) e o modelo é um ponteiro para char (string). A estrutura deverá permitir inserir, remover, atualizar, consultar a lista de carros e ordenar a lista por ano.

Sendo este tipo de serviço utilizado por vários stands, formando um sistema distribuído, torna-se imperativa a troca de informação via rede. Esta informação deve ser enviada/recebida corretamente e pode representar qualquer estrutura de dados. Neste sentido, é necessário desenvolver os devidos mecanismos de serialização/de-serIALIZAÇÃO a utilizar no envio/receção de informação.

Em suma, neste primeiro projeto pretende-se que os alunos implementem um conjunto de estruturas de dados e módulos de base na linguagem C, testando o seu funcionamento para garantir que as funcionalidades estão corretamente implementadas. Dado que estes módulos serão usados nos projetos seguintes, é importante assegurar que os mesmos não têm *bugs*.

2 Descrição Específica

O projeto 1 consiste na concretização dos seguintes módulos fundamentais:

- Criação de módulos (`data_t` e `car_t`) que definem o tipo de dados a armazenar e as operações sobre os mesmos;
- Criação de um módulo (`list`) que define uma lista encadeada simples e as operações sobre a mesma;
- Criação de um módulo (`serialization`) que define funções para serializar (codificar) uma estrutura de dados (neste caso, um *carro* ou uma *lista de carros*) numa sequência de bytes (a mensagem a enviar), e deserializar (descodificar) a mensagem para recriar a estrutura correspondente, que são necessárias para a comunicação entre máquinas.

A Figura 1 apresenta uma visão geral da relação entre os vários módulos deste projeto, os quais serão descritos em mais detalhes nas secções a seguir.

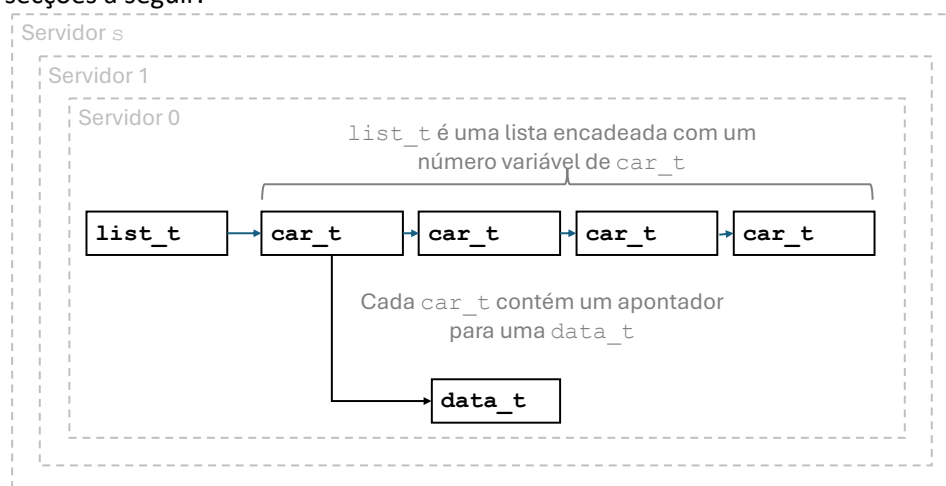


Figura 1: Visão geral da relação entre os vários módulos do projeto

Para cada um destes módulos, é fornecido um ficheiro *header* (ficheiro com extensão `.h`) com os cabeçalhos das funções. **Estes não devem ser alterados.** As concretizações das funções definidas nos ficheiros `X.h` devem ser feitas num ficheiro `X.c`, utilizando os algoritmos e métodos que o grupo achar convenientes. Se o grupo entender necessário, ou se for pedido, também pode criar um ficheiro `X-private.h` para acrescentar outras definições, cujas implementações serão também incluídas no próprio ficheiro `X.c`. Os ficheiros `.h` apresentados neste documento, bem como alguns testes para as concretizações realizadas, serão disponibilizados na página da disciplina.

Juntamente com o enunciado do projeto é disponibilizado um ficheiro ZIP contendo todos os *headers* definidos neste enunciado, bem como *templates* de ficheiros “-private.h” que deverão ser editados pelos grupos.

2.1 Definição de um bloco de dados correspondente às características do carro (*data_t*)
A primeira tarefa consiste em definir o formato dos dados que serão armazenados no servidor. Para isso, é dado o ficheiro *data.h* que define a estrutura (*struct*) que contém os blocos de dados e a respetiva dimensão, bem como funções para a sua criação, destruição e duplicação.

```
#ifndef _DATA_H
#define _DATA_H /* Módulo data */

enum marca_t
{
    MARCA_TOYOTA,
    MARCA_BMW,
    MARCA_RENAULT,
    MARCA_AUDI,
    MARCA_MERCEDES
};

enum combustivel_t
{
    COMBUSTIVEL_GASOLINA,
    COMBUSTIVEL_GASOLEO,
    COMBUSTIVEL_ELETRICO,
    COMBUSTIVEL_HIBRIDO
};

/* Estrutura que define os dados de um automóvel */
struct data_t
{
    int ano;
    float preco;
    enum marca_t marca;
    char *modelo;
    enum combustivel_t combustivel;
};

/* Função que cria um novo elemento de dados data_t.
 * Retorna a nova estrutura ou NULL em caso de erro.
 */
struct data_t *data_create(int ano, float preco, enum marca_t marca, const char *modelo, enum
combustivel_t combustivel);

/* Função que elimina um bloco de dados, libertando toda a memória por ele ocupada.
 * Retorna 0 (OK) ou -1 em caso de erro.
 */
int data_destroy(struct data_t *data);

/* Função que duplica uma estrutura data_t.
 * Retorna a nova estrutura ou NULL em caso de erro.
 */
struct data_t *data_dup(struct data_t *data);

/* Função que substitui o conteúdo de um elemento de dados data_t.
 * Retorna 0 (OK) ou -1 em caso de erro.
 */
```

```
int data_replace(struct data_t *data, int ano, float preco, enum marca_t marca, const char
*modelo, enum combustivel_t combustivel);

#endif
```

2.2 Definição da estrutura de dados lista encadeada (list_t)

O ficheiro `list.h` define as estruturas e as funções que concretizam uma lista encadeada simples [2] que armazena estruturas do tipo `car_t`.

```
#ifndef _LIST_H
#define _LIST_H /* Módulo list */

#include "data.h"

struct list_t; /* Definida em list-private.h */

/* Cria e inicializa uma nova lista de carros.
 * Retorna a lista ou NULL em caso de erro.
 */
struct list_t *list_create();

/* Elimina a lista, libertando toda a memória ocupada.
 * Retorna 0 (OK) ou -1 em caso de erro.
 */
int list_destroy(struct list_t *list);

/* Adiciona um novo carro à lista.
 * O carro é inserido na última posição da lista.
 * Retorna 0 (OK) ou -1 em caso de erro.
 */
int list_add(struct list_t *list, struct data_t *car);

/* Remove da lista o primeiro carro que corresponda ao modelo indicado.
 * Retorna 0 se encontrou e removeu, 1 se não encontrou, ou -1 em caso de erro.
 */
int list_remove_by_model(struct list_t *list, const char *modelo);

/* Obtém o primeiro carro que corresponda à marca indicada.
 * Retorna ponteiro para os dados ou NULL se não encontrar ou em caso de erro.
 */
struct data_t *list_get_by_marca(struct list_t *list, enum marca_t marca);

/* Obtém um array de ponteiros para carros de um determinado ano.
 * O último elemento do array é NULL.
 * Retorna o array ou NULL em caso de erro.
 */
struct data_t **list_get_by_year(struct list_t *list, int ano);

/* Ordena a lista de carros por ano de fabrico (crescente).
 * Retorna 0 (OK) ou -1 em caso de erro.
 */
int list_order_by_year(struct list_t *list);

/* Retorna o número de carros na lista ou -1 em caso de erro.
 */
int list_size(struct list_t *list);
```

```

/* Constrói um array de strings com os modelos dos carros na lista.
 * O último elemento do array é NULL.
 * Retorna o array ou NULL em caso de erro.
 */
char **list_get_model_list(struct list_t *list);

/* Liberta a memória ocupada pelo array de modelos.
 * Retorna 0 (OK) ou -1 em caso de erro.
 */
int list_free_model_list(char **models);

#endif

```

O ficheiro `list-private.h` especifica a relação entre a estrutura `car_t` e `data_t`, bem como a relação com a `list_t`.

```

#ifndef _LIST_PRIVATE_H
#define _LIST_PRIVATE_H

#include "data.h"

/* Nó da lista que contém um carro */
struct car_t
{
    struct data_t *data; // Dados do carro
    struct car_t *next; // Próximo nó
};

/* Estrutura da lista de carros */
struct list_t
{
    int size; // Número de carros na lista
    struct car_t *head; // Cabeça da lista
};

#endif

```

2.3 Serialização e de-serialização de estruturas de dados

Este módulo do projeto consiste em transformar uma estrutura de dados num formato que possa ser enviado pela rede (isto é, converter uma estrutura com vários elementos num formato “unidimensional”), e vice-versa. O ficheiro `serialization.h` define as funções a serem concretizadas neste módulo. De salientar que a serialização tem de transformar o valor inteiro em *network byte order* e vice-versa.

O formato dos dados serializados está indicado na descrição das funções no ficheiro `serialization.h`.

```

#ifndef _SERIALIZATION_H
#define _SERIALIZATION_H

#include "data.h"
#include "list.h"

/* Serializa um carro (data_t) para um buffer.
 * Retorna o tamanho do buffer ou -1 em caso de erro.
 */
int car_to_buffer(struct data_t *car, char **car_buf);

```

```

/* Desserializa um buffer para um carro (data_t).
 * Retorna o ponteiro para data_t ou NULL em caso de erro.
 */
struct data_t *buffer_to_car(char *car_buf);

/* Serializa uma lista de carros para um buffer.
 * Retorna o tamanho do buffer ou -1 em caso de erro.
 */
int car_list_to_buffer(struct list_t *list, char **list_buf);

/* Desserializa um buffer para uma lista de carros.
 * Retorna ponteiro para list_t ou NULL em caso de erro.
 */
struct list_t *buffer_to_car_list(char *list_buf);

#endif

```

3 Teste dos módulos desenvolvidos

A realização de testes para verificar que os módulos foram corretamente implementados e que as funções satisfazem os requisitos é muito importante, dado que os módulos serão usados nos projetos seguintes.

Assim, são disponibilizados testes que os grupos poderão usar para verificar a correção do seu código. De notar que os testes disponibilizados não são exaustivos, pelo que o facto de o código passar os testes não significa que esteja 100% correto e não tenha *bugs*. Por exemplo, aspetos como libertação de memória previamente reservada não são testados nestes testes.

Assim, os testes devem ser considerados uma ajuda, mas é da responsabilidade dos grupos assegurar a máxima correção possível do código desenvolvido e entregue.

4 Entrega

A entrega do projeto 1 tem de ser feita de acordo com as seguintes regras:

1. Colocar todos os ficheiros do projeto, bem como o ficheiro README mencionado abaixo, num ficheiro com compressão no formato ZIP. O nome do ficheiro será **SD-XX-projeto1.zip** (XX é o número do grupo).
2. Submeter o ficheiro **SD-XX-projeto1.zip** na página da disciplina no Moodle da FCUL, utilizando a atividade disponibilizada para tal. Apenas um dos elementos do grupo deve submeter, considerando-se apenas a submissão mais recente no caso de existirem várias.

O ficheiro ZIP deverá conter:

- o ficheiro **README**, onde os alunos podem incluir informações que julguem necessárias (e.g., limitações na implementação);
- diretorias adicionais, nomeadamente:
 - **include**: para armazenar os ficheiros `.h`;
 - **source**: para armazenar os ficheiros `.c`;
 - **object**: para armazenar os ficheiros objeto;
 - **binary**: para armazenar os ficheiros executáveis.
 - **tests**: para armazenar os ficheiros `.c` correspondentes aos testes.
- um ficheiro **Makefile** que permita a correta compilação de todos os ficheiros entregues. Não devem ser incluídos no ficheiro ZIP os ficheiros objeto (`.o`) ou executáveis. Os ficheiros de teste também não deverão ser incluídos no ficheiro ZIP. No momento da avaliação eles serão colocados pelos docentes dentro da diretoria **SD-XX-projeto1/tests**. Espera-se que o `Makefile` compile todos os módulos bem como todos os programas de teste. Os ficheiros executáveis resultantes devem ficar na diretoria **SD-XX-projeto1/binary** com os nomes **test_data**, **test_list**, e **test_serialization**.

Na entrega do trabalho, é ainda necessário ter em conta que:

- **Se não for incluído um Makefile, com a capacidade de compilar os ficheiros fonte e limpar os ficheiros resultantes da compilação, ou se houver erros de compilação (isto é, se não forem criados os ficheiros objeto e executáveis), o trabalho é considerado nulo.** Na página da disciplina, no Moodle, podem encontrar documentos do utilitário `make` e dos ficheiros `Makefile` (cortesia da disciplina de Sistemas Operativos).
- Todos os ficheiros entregues devem começar com um cabeçalho com três ou quatro linhas de comentários a dizer o número do grupo e o nome e número dos seus elementos.
- Os programas são testados no ambiente dos laboratórios de aulas, pelo que se recomenda que os alunos testem os seus programas neste mesmo ambiente.

O prazo de entrega é dia 5/10/2025, até às 23:59 horas.

Após esta data, a submissão do trabalho através do Moodle deixará de ser permitida.

5 Autoavaliação de contribuições

Cada aluno tem de preencher no Moodle um formulário de autoavaliação das contribuições individuais de cada elemento do grupo para o projeto. Por exemplo, se todos os elementos colaboraram de forma idêntica, bastará que todos indiquem que cada um contribuiu 33%. Aplicam-se as seguintes regras e penalizações:

- Alunos que não preencham o formulário até a data limite de entrega do projeto **sofrem uma penalização na nota de 20%.**
- Caso existam assimetrias significativas entre as respostas de cada elemento do grupo, o grupo poderá ser chamado para as explicar.
- Se as contribuições individuais forem diferentes, isso será refletido na nota de cada elemento do grupo, levando à atribuição de notas individuais diferentes.

O prazo de preenchimento desde formulário é o mesmo que a entrega do projeto (5/10/2025, até às 23:59 horas).

6 Plágios

Não é permitido aos alunos partilharem códigos com soluções, ainda que parciais, de nenhuma parte do projeto com outros alunos (nem através do Fórum da disciplina, nem por qualquer outro meio). Além disso, todos os códigos serão testados por um verificador de plágio. Caso alguma irregularidade seja encontrada, os projetos de todos os alunos envolvidos serão anulados e o caso será reportado aos órgãos responsáveis em Ciências@ULisboa.

Chamamos a atenção para o facto das plataformas generativas baseadas em Inteligência Artificial (e.g., o ChatGPT e o GitHub Co-Pilot) (1) gerarem um número limitado de soluções diferentes para o mesmo problema, (2) podem não resolver corretamente as alíneas descritas no projeto, e (3) incluem padrões característicos deste tipo de ferramenta, as quais podem vir a ser detetáveis pelos verificadores de plágio. Desta forma, recomendamos fortemente que os alunos não submetam trechos de código gerados por este tipo de ferramenta a fim de evitar riscos desnecessários.

Por fim, é responsabilidade de cada aluno garantir que a sua *home*, as suas diretorias e os seus ficheiros de código estão protegidos contra a leitura de outras pessoas (que não o utilizador dono dos mesmos). Por exemplo, se os ficheiros estiverem gravados na sua área de aluno nos servidores de Ciências@ULisboa, então todos os itens mencionados anteriormente devem ter as permissões de acesso 700. Se os ficheiros estiverem no GitHub, garantam que o conteúdo do vosso repositório não esteja visível publicamente. Caso contrário, a sua participação num eventual plágio será considerada ativa.

7 Bibliografia

[1] B. W. Kernighan, D. M. Ritchie, *C Programming Language*, 2nd Ed, Prentice-Hall, 1988.

[2] Wikipedia . *Linked List*. https://en.wikipedia.org/wiki/Linked_list.